Personalized Learning AI — Project Timeline

OpenTutor: an open-source personalized learning system that models learner mastery, detects misconceptions, adapts difficulty, and recommends the next best learning activity.

Phase 0 — Scope & Research Design (Days 1–2)

- Define target domain: CS/programming education.
- Define learner model: concept mastery, misconceptions, difficulty, confidence.
- Define research questions and success metrics.
- Create initial architecture and experiment plan.
- Create GitHub repository, issue board, contribution rules.

Phase 1 — Product Skeleton (Days 3–5)

- Build Next.js frontend shell and FastAPI backend.
- Add authentication/local learner profiles.
- Create course → topic → concept data model.
- Build first dashboard and quiz flow.
- Add PostgreSQL and Docker development environment.

Phase 2 — Baseline Learning Engine (Days 6–8)

- Create deterministic baseline recommendation logic.
- Track attempts, correctness, response time and confidence.
- Implement mastery score updates.
- Build concept prerequisite graph.
- Expose learner-state API and dashboard visualization.

Phase 3 — Dataset & Benchmark (Days 9–12)

- Create OpenTutor learner-interaction schema.
- Collect/use permitted educational material and generate validated examples.
- Create misconception labels and difficulty labels.
- Create train/validation/test splits without leakage.
- Build a fixed evaluation benchmark.

Phase 4 — AI Misconception Model (Days 13–17)

- Select a compact open language model.
- Fine-tune with LoRA/QLoRA for misconception classification/explanation.
- Train/evaluate on held-out learner answers.
- Compare base model vs fine-tuned model.

• Log experiments and publish model card.

Phase 5 — Personalized Learning Model (Days 18–23)

• Build a learner-state representation from interaction history.

• Implement mastery estimation and forgetting/review signals.

• Train a next-question/difficulty recommendation model.

• Compare heuristic, ML and personalized policies.

• Add offline simulation using historical learner trajectories.

Phase 6 — Adaptive Tutor (Days 24–27)

• Combine misconception detection + mastery estimation + recommendation.

• Generate targeted explanations and follow-up questions.

• Prevent repeated exposure to mastered concepts.

• Introduce prerequisite-aware remediation.

• Add confidence and uncertainty handling.

Phase 7 — Full Frontend (Days 28–31)

• Learning map / knowledge graph.

• Mastery by concept and topic.

• Personalized 'What should I study next?' view.

• Adaptive quiz interface.

• Misconception explanations and progress history.

Phase 8 — Evaluation & Ablations (Days 32–35)

• Measure recommendation accuracy and learning gains.

• Evaluate misconception classification.

• Run ablations: no learner model, no misconception model, no prerequisite graph.

• Compare static vs adaptive sequencing.

• Create charts, tables and reproducible experiment reports.

Phase 9 — Deployment & Open Source (Days 36–39)

• Dockerize services.

• Deploy frontend and inference/API services.

• Publish model and dataset artifacts.

• Add CI, tests, linting and security basics.

• Write documentation, contribution guide and demo instructions.

Phase 10 — Polish & Release (Days 40–42)

- Run end-to-end testing.
- Improve latency and inference cost.
- Create demo dataset and seeded learner account.
- Record demo and prepare project README.
- Tag v1.0 and publish a technical report.

Recommended Research Milestones

Milestone	Evidence
M1: Baseline	Rule-based learner model works end-to-end
M2: Dataset	Versioned dataset + leakage-safe benchmark
M3: Model	Fine-tuned misconception model beats baseline
M4: Personalization	Recommendation model beats static sequencing
M5: Adaptive Tutor	End-to-end learner state changes future content
M6: Evaluation	Ablation study demonstrates which components matter
M7: Release	Live demo + GitHub + model/dataset + reproducible training

Important: The timeline is a target, not a promise. Training time will depend on dataset size, GPU availability, model size and experiment count.